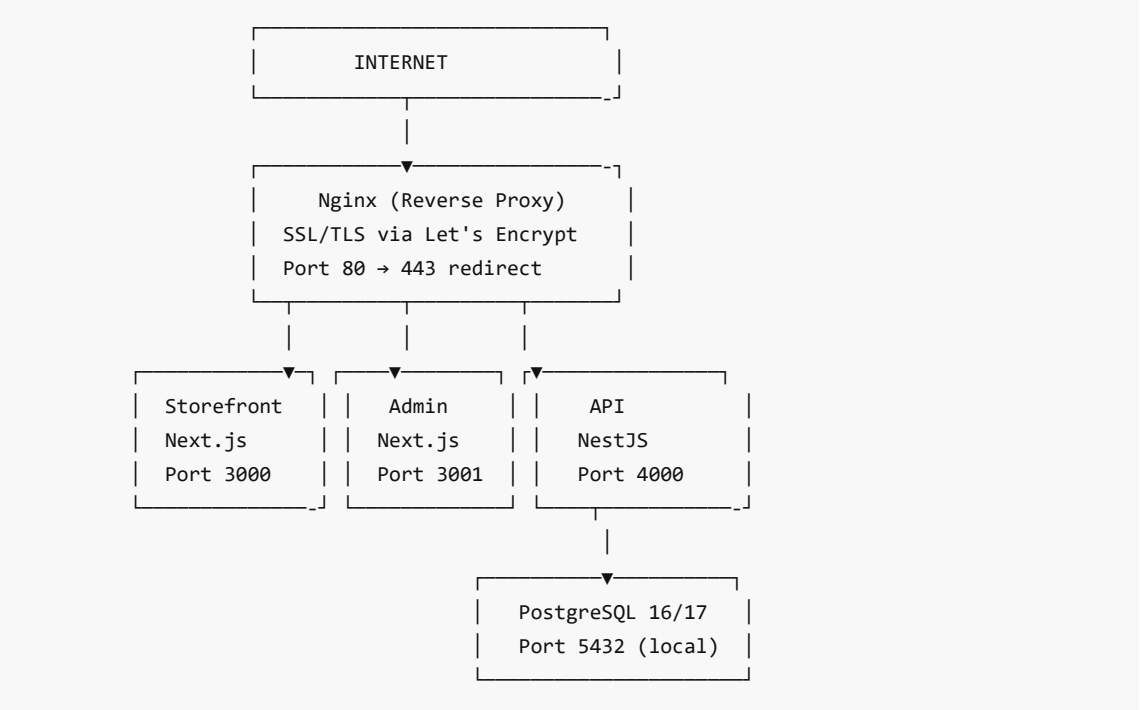
Naro Fashion — Production Deployment Guide

Overview

This guide covers the complete deployment of the Naro Fashion multi-tenant SaaS e-commerce platform from scratch to a production server.

Architecture:



URLs:

- `narofashion.co.tz` → Storefront (customer-facing shop)
- `admin.narofashion.co.tz` → Admin dashboard
- `api.narofashion.co.tz` → REST API

Prerequisites

- A domain name (e.g., `narofashion.co.tz`)
- A VPS with at least 2GB RAM (Vultr, Hetzner, DigitalOcean, Linode)
- A GitHub account with the repo: `github.com/rjurio/naro-fashion`
- A credit/debit card for VPS billing

Step 1: Provision a VPS

Option A: Vultr (Recommended)

1. Go to https://www.vultr.com and create an account
2. Click **Deploy +** → **Cloud Compute** → **Shared CPU**
3. Configure:

Setting	Value
Location	Frankfurt, DE (closest to East Africa)
Image	Ubuntu 24.04 LTS
Plan	vhf-1c-2gb ($12/mo) or vc2-2c-2gb ($15/mo)
SSH Key	Add your public key (optional)
Hostname	`naro-fashion-prod`

4. Click **Deploy**
5. Note the **IP address** and **root password** from the server details page

Option B: Hetzner ($7.50/mo)

1. Go to https://console.hetzner.com
2. Requires ID verification (passport upload)
3. Create project → Add Server → Ubuntu 24.04, CX32, Falkenstein

Option C: DigitalOcean ($12/mo)

1. Go to https://www.digitalocean.com
2. Create Droplet → Ubuntu 24.04, Basic, Regular $12/mo

Step 2: Connect to the Server

Generate SSH Key (if you don't have one)

```
# On your local machine (Git Bash, Mac Terminal, or Linux)
ssh-keygen -t ed25519 -C "your@email.com"
cat ~/.ssh/id_ed25519.pub
# Copy the output and add to server
```

Connect via SSH

```
ssh root@<YOUR_VPS_IP>
# Enter password if no SSH key was added
```

Add SSH Key to Server (if using password login)

```
# On the server:
echo "ssh-ed25519 AAAA... your@email.com" >> ~/.ssh/authorized_keys
```

Step 3: Install Server Dependencies

Run all commands on the server as root:

```
# Update system
apt update && apt upgrade -y

# Install essentials
apt install -y curl git build-essential nginx certbot python3-certbot-nginx ufw

# Install Node.js 22 LTS
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt install -y nodejs

# Install pnpm (package manager)
npm install -g pnpm@10

# Install PM2 (process manager)
npm install -g pm2

# Install PostgreSQL
apt install -y postgresql postgresql-contrib
systemctl enable postgresql
systemctl start postgresql

# Verify installations
node -v    # Should show v22.x
pnpm -v    # Should show 10.x
pm2 -v     # Should show 6.x
psql --version  # Should show 16.x
```

Step 4: Configure PostgreSQL

```
# Generate a strong password
openssl rand -base64 32

# Create database and user
sudo -u postgres psql
```

Inside PostgreSQL:

```
CREATE USER naro_admin WITH PASSWORD '<YOUR_STRONG_PASSWORD>';
CREATE DATABASE naro_fashion OWNER naro_admin;
GRANT ALL PRIVILEGES ON DATABASE naro_fashion TO naro_admin;
\q
```

Save the password — you'll need it for the `.env` file.

Step 5: Clone the Repository

```
mkdir -p /var/www/naro-fashion
cd /var/www/naro-fashion
git clone https://github.com/rjurio/naro-fashion.git .
```

Step 6: Configure Environment Variables

Main `.env` file

```
cat > /var/www/naro-fashion/.env << 'EOF'
NODE_ENV=production

DATABASE_URL="postgresql://naro_admin:<DB_PASSWORD>@localhost:5432/naro_fashion?
schema=public"

JWT_SECRET="<GENERATE: openssl rand -base64 64>"
JWT_REFRESH_SECRET="<GENERATE: openssl rand -base64 64>"
JWT_EXPIRATION="15m"
JWT_REFRESH_EXPIRATION="7d"

STOREFRONT_URL="https://narofashion.co.tz"
ADMIN_URL="https://admin.narofashion.co.tz"
API_URL="https://api.narofashion.co.tz"
NEXT_PUBLIC_API_URL="https://api.narofashion.co.tz/api/v1"

SMTP_HOST=""
SMTP_PORT="587"
SMTP_USER=""
SMTP_PASS=""
SMTP_FROM="noreply@narofashion.co.tz"

INSTAGRAM_BUSINESS_ACCOUNT_ID="17841418108905851"
FACEBOOK_APP_ID="4338851449722487"
EOF
```

Replace `<DB_PASSWORD>` with your PostgreSQL password, and generate JWT secrets:

```
openssl rand -base64 64 | tr -dc 'a-zA-Z0-9' | head -c 64
```

Copy env to sub-packages

```
cp .env packages/database/.env
cp .env apps/api/.env

# Storefront env
echo 'NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1' > apps/storefront/.env.local
echo 'NEXT_PUBLIC_TENANT_SLUG=naro-fashion' >> apps/storefront/.env.local
```

```
# Admin env
echo 'NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1' > apps/admin/.env.local
```

Step 7: Install Dependencies & Build

```
cd /var/www/naro-fashion

# Install all dependencies
pnpm install

# Install multer (required for file uploads)
pnpm add multer @types/multer --filter api

# Generate Prisma client
cd packages/database && npx prisma generate && cd ../..

# Push schema to database
cd packages/database && npx prisma db push --accept-data-loss && cd ../..

# Run multi-tenant migration (creates first tenant + platform admin)
node packages/database/prisma/migrate-to-multi-tenant.js

# Seed tenant data (settings, CMS pages, payment methods)
node packages/database/prisma/seed-tenant.js

# Build all 3 apps
pnpm build
```

Expected build output: `Tasks: 3 successful, 3 total`

If build fails with prerender errors, add `export const dynamic = "force-dynamic";` to the root layout of both Next.js apps.

Step 8: Configure PM2 (Process Manager)

The `ecosystem.config.js` file is already in the repo. For standalone builds:

```
cat > /var/www/naro-fashion/ecosystem.config.js << 'EOF'
module.exports = {
  apps: [
    {
      name: 'naro-api',
      cwd: '/var/www/naro-fashion/apps/api',
      script: 'dist/main.js',
      instances: 1,
      env: { NODE_ENV: 'production', PORT: 4000 },
    },
    {
      name: 'naro-storefront',
```

```
      cwd: '/var/www/naro-fashion/apps/storefront/.next/standalone/apps/storefront',
      script: 'server.js',
      instances: 1,
      env: { NODE_ENV: 'production', PORT: 3000, HOSTNAME: '0.0.0.0' },
    },
    {
      name: 'naro-admin',
      cwd: '/var/www/naro-fashion/apps/admin/.next/standalone/apps/admin',
      script: 'server.js',
      instances: 1,
      env: { NODE_ENV: 'production', PORT: 3001, HOSTNAME: '0.0.0.0' },
    },
  ],
};
EOF
```

Copy static assets for standalone mode

```
# Storefront static files
cp -r apps/storefront/.next/static
apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public

# Admin static files
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public
```

Start all apps

```
cd /var/www/naro-fashion
pm2 start ecosystem.config.js
pm2 save
pm2 startup   # Follow the printed command to enable auto-start
```

Verify all apps are running

```
pm2 list
# All 3 should show "online" status
```

Step 9: Configure Nginx

```
cat > /etc/nginx/sites-available/narofashion << 'NGINX'
# Storefront
server {
    listen 80;
    server_name narofashion.co.tz www.narofashion.co.tz;
```

```
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# Admin
server {
    listen 80;
    server_name admin.narofashion.co.tz;
    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# API
server {
    listen 80;
    server_name api.narofashion.co.tz;
    client_max_body_size 30M;
    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
    location /uploads/ {
        proxy_pass http://127.0.0.1:4000/uploads/;
        add_header Cache-Control "public, max-age=604800";
    }
}
NGINX
```

```
# Enable site and test
ln -sf /etc/nginx/sites-available/narofashion /etc/nginx/sites-enabled/
rm -f /etc/nginx/sites-enabled/default
nginx -t
systemctl reload nginx
```

Step 10: Configure DNS (Habari Node)

The domain `narofashion.co.tz` is registered with **Habari Node** (habarinode.co.tz / habari.co.tz), a Tanzanian ISP and domain registrar accredited by tzNIC.

10a. Login to Habari Node Client Portal

1. Go to **https://hosting.habari.co.tz/**
2. Login with your Habari Node account credentials
3. Navigate to **Domains** → select `narofashion.co.tz`
4. Click **DNS Management** or **Zone Editor** or **Manage DNS**

> *If Habari Node uses cPanel/WHM (common for Tanzanian hosts), look for: **Domains** → **Zone Editor** in the cPanel sidebar*

10b. Add DNS A Records

Add these 4 records (delete any existing A records pointing elsewhere first):

Type	Name/Host	Value/Points to	TTL
A	`@`	`80.240.30.107`	3600
A	`www`	`80.240.30.107`	3600
A	`admin`	`80.240.30.107`	3600
A	`api`	`80.240.30.107`	3600

Notes:

- `@` means the root domain (`narofashion.co.tz`). Some registrars show this as blank or the full domain name
- For `www` , `admin` , `api` — enter just the subdomain name (not `www.narofashion.co.tz`)
- Type must be **A** (not CNAME, not MX)
- If TTL options are limited, choose the lowest available (300, 600, or 3600)

10c. If Habari Node Uses Nameservers Instead of Direct DNS

Some registrars don't provide a DNS zone editor. Instead, you point nameservers to a DNS provider. In that case:

Option A: Use Vultr's DNS (free)

1. In Vultr dashboard → **DNS** → **Add Domain** → enter `narofashion.co.tz`
2. Add the 4 A records above in Vultr's DNS panel
3. In Habari Node, change nameservers to:
    - `ns1.vultr.com`

- `ns2.vultr.com`

Option B: Use Cloudflare (free)

1. Go to https://dash.cloudflare.com → **Add Site** → enter `narofashion.co.tz`
2. Select **Free** plan
3. Add the 4 A records above in Cloudflare's DNS panel
4. In Habari Node, change nameservers to the ones Cloudflare provides (e.g., `xxx.ns.cloudflare.com`)

10d. If You Need Help from Habari Node

Contact Habari Node support:

- **Portal**: https://hosting.habari.co.tz/ (open a support ticket)
- **Email**: software@habari.co.tz
- **Phone**: +255 659 074 444
- **Office**: Arusha, Tanzania

Tell them: "*I need to add A records for narofashion.co.tz pointing to IP 80.240.30.107. I need records for @, www, admin, and api subdomains.*"

10e. Verify DNS Propagation

Wait 5-30 minutes (up to 24 hours for nameserver changes), then test:

```
# From your terminal
dig narofashion.co.tz +short
# Should return: 80.240.30.107

dig admin.narofashion.co.tz +short
# Should return: 80.240.30.107

dig api.narofashion.co.tz +short
# Should return: 80.240.30.107
```

Or use https://dnschecker.org to check propagation globally.

Step 11: Install SSL Certificates

After DNS is pointing correctly:

```
certbot --nginx \
  -d narofashion.co.tz \
  -d www.narofashion.co.tz \
  -d admin.narofashion.co.tz \
  -d api.narofashion.co.tz \
  --email hello@narofashion.co.tz \
  --agree-tos \
  --non-interactive

# Verify auto-renewal
certbot renew --dry-run
```

Step 12: Set Tenant Domain

Connect the production domain to the tenant in the database:

```
# Via psql (most reliable method)
sudo -u postgres psql -d naro_fashion -c "UPDATE \"Tenant\" SET \"domain\" =
'narofashion.co.tz' WHERE \"slug\" = 'naro-fashion';"

# Restart storefront so middleware picks up the domain
pm2 restart naro-storefront
```

> ***Note:*** *The* `node -e` *approach may fail due to pnpm module resolution. Using* `psql` *directly is more reliable on the server.*

Step 13: Configure Firewall

```
ufw allow OpenSSH
ufw allow 'Nginx Full'
ufw enable
ufw status
```

Step 14: Set Up Database Backups

```
# Create backup directory
mkdir -p /var/backups

# Add daily backup cron (runs at 2 AM)
crontab -e
# Add these lines:
0 2 * * * pg_dump -U naro_admin naro_fashion | gzip > /var/backups/naro_fashion_$(date
+\%F).sql.gz
0 3 * * * find /var/backups -name "naro_fashion_*.sql.gz" -mtime +30 -delete
```

Step 15: Set Up Log Management

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
```

Step 16: Set Up CI/CD (Auto-Deploy on Push to prod)

16a. Create prod branch

```
# On your local machine
cd /path/to/naro-fashion
git branch prod
git push origin prod
```

16b. Generate SSH key on the server

```
# On the VPS
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N '' -C 'deploy@naro-fashion-prod'
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
```

16c. Create GitHub Actions workflow

Create `.github/workflows/deploy-prod.yml`:

```
name: Deploy to Production

on:
  push:
    branches: [prod]

jobs:
  deploy:
    name: Deploy to VPS
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.VPS_HOST }}
          username: ${{ secrets.VPS_USER }}
          key: ${{ secrets.VPS_SSH_KEY }}
          port: 22
          script_stop: true
          script: |
            cd /var/www/naro-fashion
            chmod +x deploy.sh
            bash deploy.sh
```

16d. Set GitHub Secrets

Using GitHub CLI (requires `gh` authenticated):

```
echo "80.240.30.107" | gh secret set VPS_HOST
echo "root" | gh secret set VPS_USER

# Get the private key from the server and set it:
```

```
ssh root@80.240.30.107 "cat ~/.ssh/id_ed25519" | gh secret set VPS_SSH_KEY

# Verify:
gh secret list
```

Or set manually: GitHub repo → Settings → Secrets and variables → Actions → New repository secret.

16e. Deploy Script on Server

The deploy script at `/var/www/naro-fashion/deploy.sh` backs up env files before `git reset --hard`:

```
#!/bin/bash
set -e

echo "🚀 Deploying Naro Fashion..."
cd /var/www/naro-fashion

# Backup env files (git reset --hard will delete them)
cp apps/storefront/.env.local /tmp/storefront.env.local 2>/dev/null || true
cp apps/admin/.env.local /tmp/admin.env.local 2>/dev/null || true
cp .env /tmp/naro.env 2>/dev/null || true
cp apps/api/.env /tmp/api.env 2>/dev/null || true

# Pull latest from prod branch
git fetch origin prod
git reset --hard origin/prod

# Restore env files
cp /tmp/storefront.env.local apps/storefront/.env.local 2>/dev/null || true
cp /tmp/admin.env.local apps/admin/.env.local 2>/dev/null || true
cp /tmp/naro.env .env 2>/dev/null || true
cp /tmp/api.env apps/api/.env 2>/dev/null || true
cp .env packages/database/.env 2>/dev/null || true

# Install dependencies
pnpm install

# Generate Prisma client & push schema
cd packages/database
npx prisma generate
npx prisma db push --accept-data-loss
cd ../..

# Build all apps
pnpm build

# Copy static files for standalone Next.js
cp -r apps/storefront/.next/static
apps/storefront/.next/standalone/apps/storefront/.next/static 2>/dev/null || true
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public
2>/dev/null || true
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
```

```
2>/dev/null || true
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public 2>/dev/null || true

# Restart PM2 processes
pm2 restart ecosystem.config.js
pm2 save

echo "✅ Deployment complete! $(date)"
```

16f. How to Deploy

```
# 1. Work on master branch normally
git add . && git commit -m "your changes"
git push origin master

# 2. When ready to deploy to production:
git checkout prod
git merge master --no-edit
git push origin prod    # ← Triggers automatic deployment via GitHub Actions
git checkout master

# 3. Monitor deployment:
gh run list --limit 1
gh run view <run-id> --log   # If failed, check logs
```

16g. Key CI/CD Notes

- `.env.local` files are NOT in git — the deploy script backs them up before `git reset --hard` and restores after
- `multer` must be in `apps/api/package.json` dependencies (not just devDependencies) — `pnpm install` doesn't auto-resolve hoisted packages
- Both Next.js apps need `output: 'standalone'`, `typescript: { ignoreBuildErrors: true }` in `next.config.js`
- Root `app/layout.tsx` in both apps needs `export const dynamic = "force-dynamic"` to prevent prerender errors
- Static files must be copied to standalone directory after each build

---

Step 17: Add Swap Space (Recommended for 2GB RAM)

```
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab
free -h   # Verify swap is active
```

Post-Deployment Checklist

#	Task	How to Verify
1	Storefront loads	Visit `https://narofashion.co.tz`
2	Admin loads	Visit `https://admin.narofashion.co.tz`
3	API responds	Visit `https://api.narofashion.co.tz/api/docs`
4	SSL working	Check padlock icon in browser
5	Admin login	`admin@narofashion.co.tz` / `admin123`
6	Platform login	`platform@naro.co.tz` / `Admin123`
7	**CHANGE PASSWORDS**	Change both admin + platform admin passwords immediately
8	Products visible	Browse storefront, check images load
9	Mobile works	Open site on phone
10	Backups running	`ls /var/backups/` after midnight
11	Swap enabled	`free -h` shows swap
12	PM2 auto-start	`sudo reboot` then verify apps come back

Updating the Application

Manual Update

```
cd /var/www/naro-fashion
git pull origin master
pnpm install
cd packages/database && npx prisma generate && npx prisma db push --accept-data-loss && cd
../..
pnpm build

# Copy static files for standalone mode
cp -r apps/storefront/.next/static
apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public

pm2 restart all
```

Using the Deploy Script

```
chmod +x deploy.sh
./deploy.sh
```

Monitoring

```
pm2 monit          # Live CPU/memory monitor
pm2 logs           # Stream all logs
pm2 logs naro-api  # Stream API logs only
pm2 list           # Check status of all apps
pm2 info naro-api  # Detailed info for one app
```

Troubleshooting

App won't start (PM2 shows "errored")

```
pm2 logs <app-name> --lines 50   # Check error logs
```

502 Bad Gateway

- App hasn't started yet — wait 10-15 seconds after restart and retry
- Check `pm2 list` to see if apps are online (status = "online")
- If API keeps restarting (high restart count), check logs: `pm2 logs naro-api --lines 30`

"Cannot find module" errors on API

```
# Common: multer not installed
cd /var/www/naro-fashion && pnpm add multer @types/multer --filter api
pm2 restart naro-api
```

Database connection failed

```
sudo -u postgres psql -c "SELECT 1"    # Test PostgreSQL is running
systemctl status postgresql             # Check service status
systemctl restart postgresql            # Restart if needed
```

Run raw SQL on production database

```
sudo -u postgres psql -d naro_fashion   # Opens psql shell
# Then run SQL queries, e.g.:
# SELECT * FROM "Tenant";
# \q to exit
```

SSL certificate issues

```
certbot renew --force-renewal           # Force renew certificates
nginx -t && systemctl reload nginx      # Reload Nginx
certbot certificates                    # List all certificates + expiry dates
```

Next.js build fails with prerender errors

Both Next.js apps need these settings in `next.config.js`:

```
typescript: { ignoreBuildErrors: true },
eslint: { ignoreDuringBuilds: true },
output: 'standalone',
```

And root `app/layout.tsx` needs at top:

```
export const dynamic = "force-dynamic";
```

Client pages (`"use client"`) also need `export const dynamic = "force-dynamic";` AFTER the `"use client"` directive.

Standalone build — static files missing (blank pages)

After every `pnpm build`, you must copy static files:

```
cp -r apps/storefront/.next/static
apps/storefront/.next/standalone/apps/storefront/.next/static
cp -r apps/storefront/public apps/storefront/.next/standalone/apps/storefront/public
cp -r apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -r apps/admin/public apps/admin/.next/standalone/apps/admin/public
```

Out of memory during build

```
# Check memory
free -h

# If no swap, add it:
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab

# Or build on your local machine and deploy the built files
```

Nginx returns wrong site for a domain

```
nginx -t                              # Test config syntax
cat /etc/nginx/sites-enabled/narofashion  # Verify server_name entries
systemctl reload nginx                # Apply changes
```

Known Build & Deployment Issues

These are issues discovered during the actual deployment:

Issue	Root Cause	Fix

`Cannot find module 'multer'`	pnpm strict hoisting doesn't resolve multer for API	`pnpm add multer @types/multer --filter api`
`Module not found: '@/types/model-viewer'`	`.d.ts` files can't be imported as modules	Change `import '@/types/...'` to `/// <reference path="..." />`
`The "use client" directive must be placed before other expressions`	`export const dynamic` added before `"use client"`	`dynamic` export must come AFTER `"use client"`
`Error occurred prerendering page "/auth/login"`	Pages use `localStorage`/browser APIs during SSR prerender	Add `export const dynamic = "force-dynamic"` to root layout
`Property 'imageUrl' does not exist on type 'Category'`	TypeScript strict mode catches missing interface fields	Add missing field to interface or disable TS checks in build
`Cannot find module '@prisma/client'` when running scripts from `/tmp`	pnpm node_modules not in `/tmp` resolve path	Run scripts from project directory or use `psql` for DB queries
Standalone Next.js shows blank page	Static assets not copied to standalone directory	Copy `.next/static` and `public` after each build

---

Infrastructure Reference

Current Production Setup

Component	Details
VPS Provider	Vultr (vultr.com)
Server Plan	vhf-1c-2gb (1 vCPU, 2GB RAM, 64GB NVMe SSD)
Server Location	Frankfurt, Germany
Server IP	`80.240.30.107`
OS	Ubuntu 24.04 LTS
Domain Registrar	Habari Node (habarinode.co.tz / hosting.habari.co.tz)
DNS Provider	Vultr DNS (ns1.vultr.com, ns2.vultr.com)
SSL	Let's Encrypt (auto-renews, expires 2026-06-18)
Node.js	v22.x LTS
Package Manager	pnpm v10.x
Process Manager	PM2
Web Server	Nginx
Database	PostgreSQL 16
App Directory	`/var/www/naro-fashion`

Port Mapping

Domain	Nginx →	App
`narofashion.co.tz`	`127.0.0.1:3000`	Storefront (Next.js standalone)
`admin.narofashion.co.tz`	`127.0.0.1:3001`	Admin (Next.js standalone)
`api.narofashion.co.tz`	`127.0.0.1:4000`	API (NestJS dist)

DNS Records (Vultr DNS)

Type	Name	Value	TTL
A	`@`	`80.240.30.107`	300
A	`www`	`80.240.30.107`	3600
A	`admin`	`80.240.30.107`	3600
A	`api`	`80.240.30.107`	3600
CNAME	`*`	`narofashion.co.tz`	300
NS		`ns1.vultr.com`	300
NS		`ns2.vultr.com`	300

Default Credentials (CHANGE IMMEDIATELY)

Login	Email	Password	URL
Tenant Admin	`admin@narofashion.co.tz`	`admin123`	`https://admin.narofashion.co.tz`
Platform Admin	`platform@naro.co.tz`	`Admin123`	`https://admin.narofashion.co.tz/platform-login`

Cost Summary

Service	Monthly Cost	Annual Cost
VPS (Vultr vhf-1c-2gb)	$12.00	$144.00
Domain (.co.tz via Habari Node)	~$1.76	TZS 21,186/yr (~$8.25)
SSL (Let's Encrypt)	Free	Free
Email (Brevo free tier)	Free	Free
DNS (Vultr DNS)	Free	Free
Total	**~$13.76/mo**	**~$152.25/yr**

Security Hardening

Immediate Actions (after deployment)

1. **Change default passwords** for admin and platform admin
2. **Enable SSH key-only login** — disable password authentication:

```
# Edit SSH config
nano /etc/ssh/sshd_config
# Set: PasswordAuthentication no
# Set: PermitRootLogin prohibit-password
systemctl restart sshd
```

Already Configured

- **Firewall (UFW)**: Only ports 22 (SSH), 80 (HTTP), 443 (HTTPS) open
- **PostgreSQL**: Listens only on localhost (not exposed to internet)
- **Helmet**: CSP, XSS protection, clickjacking prevention enabled on API
- **Rate Limiting**: 100 requests per 60 seconds per IP (NestJS Throttler)
- **Account Lockout**: 5 failed login attempts → 30-minute lock
- **Password Hashing**: bcryptjs with salt round 12
- **JWT Secrets**: 64-byte random strings, unique to this deployment
- **CORS**: Only storefront and admin URLs whitelisted
- **File Upload Limits**: 5MB images, 25MB 3D models

Recommended Future Improvements

- Set up **fail2ban** for SSH brute-force protection
- Add **Cloudflare** as CDN/DDoS protection layer
- Enable **PostgreSQL SSL** connections
- Set up **offsite backup** (e.g., S3-compatible storage)
- Configure **monitoring alerts** (PM2 + uptime monitoring service)

Adding a New Tenant Client

When selling the platform to a new client:

1. **Login** as platform admin at `https://admin.narofashion.co.tz/platform-login`
2. Go to **Tenants** → **New Tenant**
3. Fill in: company name, slug, admin email/password, subscription plan
4. The new tenant gets their own isolated storefront, admin, and data

To connect a **custom domain** for the new tenant:

1. Add their domain as an A record pointing to `80.240.30.107` in Vultr DNS
2. Add SSL: `certbot --nginx -d newclient.co.tz`
3. Update tenant domain: `sudo -u postgres psql -d naro_fashion -c "UPDATE \"Tenant\" SET \"domain\" = 'newclient.co.tz' WHERE \"slug\" = 'new-client-slug';"`
4. Restart storefront: `pm2 restart naro-storefront`

Naro Fashion — Multi-Tenant SaaS E-Commerce Platform Deployed on Vultr VPS • Frankfurt, Germany Server IP: 80.240.30.107 Domain: narofashion.co.tz (registered via Habari Node, DNS via Vultr) Last updated: March 2026